



# Book 2: Engineering and Architecture



Cynqra Engineering Book, v0.1. This book answers: how do we build it? Decisions, schemas, contracts, security. Owner: CTO. Concepts are defined in Book 0. Scope is defined in Book 1. This book defines the fields, the decisions and the defenses.

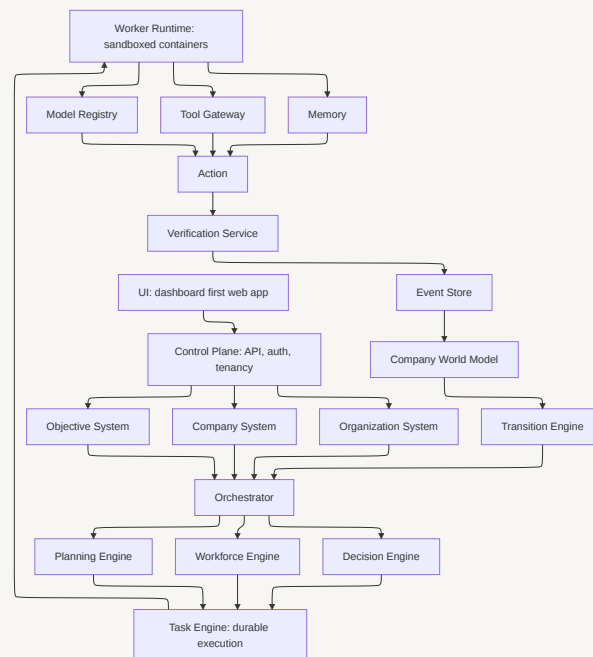
- [1. Architecture principles](#)
- [2. System map](#)
- [3. Build vs buy](#)
- [4. Architecture Decision Records](#)
- [5. Entity schema v0](#)
- [6. Event model](#)
- [7. Protocol contracts \(MVP subset\)](#)
- [8. Security and sandbox model](#)
- [9. Spike plan: de-risk in weeks 1 and 2](#)
- [10. Decision labels and the eval data strategy](#)

## 1. Architecture principles

- 1. Event sourced core.** Every meaningful change is an immutable event (Book 0 §17). State is derived, history is never lost, replay is possible.
- 2. Policy gated execution.** No worker action executes without passing the authority engine (Book 0 §12).

3. **Sandbox first runtime.** Workers act inside isolated, disposable environments with allowlisted tools.
4. **Model independence.** All intelligence access goes through the model registry. No model SDK calls outside it (Book 0 §9).
5. **Build the org layer, buy everything else.** Our IP is worker identity, authority, verification and transitions. Auth, billing, queues and workflow plumbing are commodities.

## 2. System map



Cross cutting: Policy Engine (pre action), Budget Service (circuit breaker), Audit Log (append only).

## 3. Build vs buy

| Domain                        | Decision                          | Notes     |
|-------------------------------|-----------------------------------|-----------|
| Auth, billing, org management | <b>Buy</b> (managed auth, Stripe) | Commodity |

| Domain                                                                               | Decision                     | Notes                                         |
|--------------------------------------------------------------------------------------|------------------------------|-----------------------------------------------|
| Durable task execution                                                               | <b>Buy</b> (workflow engine) | Tasks survive crashes, retries, scheduling    |
| Primary store                                                                        | <b>Buy</b> (Postgres)        | Plus an append only event table, see ADR-2    |
| Orchestrator, Worker Runtime, Policy Engine, Verification, Model Registry, Org Graph | <b>Build</b>                 | This is Cynqra                                |
| Model inference                                                                      | <b>Buy</b> (multi provider)  | The registry abstracts providers from day one |

## 4. Architecture Decision Records

Status:  proposed,  decided,  deferred.

| #     | Decision                                                      | Leaning                                                                                                 | Status                                                                                |
|-------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ADR-1 | Orchestration: custom event driven vs durable workflow engine | Durable workflow engine for the Task Engine, custom event bus for protocols                             |  |
| ADR-2 | State storage                                                 | Postgres plus append only event log (envelope below), derived read models                               |  |
| ADR-3 | Worker runtime isolation                                      | One disposable container per worker session, no shared credentials, network egress allowlist            |  |
| ADR-4 | Model routing                                                 | Registry with per task type routing rules, fallback chain, version pinning, regression gate (Book 0 §9) |  |
| ADR-5 | Tool execution                                                | Tool Gateway: allowlisted tools, scoped short lived tokens, per action policy check                     |  |

| #      | Decision                    | Leaning                                                                                                   | Status |
|--------|-----------------------------|-----------------------------------------------------------------------------------------------------------|--------|
| ADR-6  | Policy and authority engine | Declarative policies (role × risk tier × action), evaluated pre action, every denial logged               | ●      |
| ADR-7  | Verification service        | Tiered: automated checks → independent second model review → human approval. Verdict recorded as an event | ●      |
| ADR-8  | Memory and context          | Layered context per Book 0 §19, retrieval per task. Workers never receive full company dumps              | ●      |
| ADR-9  | Frontend                    | Dashboard first web app (Book 0 §21). Chat is a secondary surface only                                    | ●      |
| ADR-10 | Multi tenancy               | Shared schema with company_id on every row plus strict row level security. Revisit at scale               | ●      |
| ADR-11 | Secrets                     | Central vault. Secrets injected at runtime, never present in worker context windows                       | ●      |
| ADR-12 | Prompt injection defense    | See §8: untrusted content isolation, action gating, exfiltration controls                                 | ●      |

## 5. Entity schema v0

| Entity    | Core fields                                                                          |
|-----------|--------------------------------------------------------------------------------------|
| Company   | id, founder_id, name, stage, autonomy_level, budget_cap, created_at                  |
| Objective | id, company_id, statement, structured (jsonb), priority, constraints (jsonb), status |

| Entity     | Core fields                                                                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Worker     | id, company_id, role, capabilities[], intelligence_source_id, authority_policy_id, cost_profile, status                                                       |
| Task       | id, workstream_id, owner_worker_id, objective_ref, inputs, expected_output, dependencies[], tools[], budget, deadline, verification_method, risk_tier, status |
| Decision   | id, company_id, problem, evidence_refs[], recommendation, cost, risk, confidence, status (pending / approved / rejected), resolved_by, resolved_at            |
| Transition | id, company_id, problem, evidence_refs[], proposed_change, expected_result, cost, risk, approved_by, actual_result, confidence                                |
| Event      | envelope below                                                                                                                                                |

## 6. Event model

**Envelope:** `id · company_id · type · actor (worker / founder / system) · authority_snapshot · context_refs[] · payload · created_at`

**Catalog v0:** `company.created · objective.created · objective.changed · state.changed · worker.hired · worker.assigned · worker.reassigned · worker.replaced · task.created · task.started · task.blocked · task.failed · task.completed · task.verified · decision.created · decision.approved · decision.rejected · budget.changed · organization.changed · transition.proposed · transition.approved · transition.executed · outcome.recorded`

## 7. Protocol contracts (MVP subset)

All protocol messages are structured objects, not free chat (cost and auditability, Book 1 §7).

- **Blocker:** `{ raised_by, task_id, category, severity, description, needs_from }`
- **Handoff:** `{ from_worker, to_worker, task_id, artifacts[], context_ref, acceptance_check }`
- **Escalation:** `{ raised_by, issue, severity, owner, required_action, status }`

- **Approval:** `{ decision_id, recommendation, evidence_refs[], cost, risk, confidence, options: approve, reject, request_evidence }`

Deferred to Phase 2: Request, Review, Disagreement, Decision (as a protocol, distinct from the Decision entity), Override, Failure, Reassignment.

## 8. Security and sandbox model

1. **Identity and audit.** Every action carries identity, permission, authority, context and an audit record (Book 0 §20).
2. **Sandboxing.** Disposable containers, filesystem scoped to the workspace, egress allowlist, no host access.
3. **Prompt injection defense.** All external content (web, email, customer feedback, files) is labeled untrusted. Untrusted content can never directly trigger a tool action. Actions derived from untrusted content require a policy re-check. Secrets never enter contexts that read untrusted content.
4. **Blast radius containment.** Per worker action rate limits, per company budget circuit breaker (Book 1 P0 #10), a global kill switch that freezes all workers mid task, and all HIGH risk actions are human gated regardless of autonomy level.
5. **Rollback.** Code via git, infra via IaC, workforce and config changes via stored prior state (Book 0 §14).

## 9. Spike plan: de-risk in weeks 1 and 2

| Spike                     | Question                                                                          | Success criterion                                                 | Timebox |
|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|---------|
| S1: Objective structuring | Can an LLM reliably convert a messy founder objective into our structured schema? | 8 of 10 test objectives structure correctly with one retry        | 3 days  |
| S2: Coordination cost     | What does a simulated 4 worker org cost per task in tokens and latency?           | Cost model lands within 2x of estimate, protocols stay structured | 4 days  |

| Spike                    | Question                                                       | Success criterion                                       | Timebox |
|--------------------------|----------------------------------------------------------------|---------------------------------------------------------|---------|
| S3: Verification quality | Does tiered verification catch bad output, including non code? | At least 80% of seeded defects caught before "verified" | 4 days  |

Each spike ends with a one page result memo appended to this book. A failed spike is a successful spike. It means we learned cheaply.

## 10. Decision labels and the eval data strategy

Raised in the bench review: every founder action in the Decision inbox is a training label, and every recommendation must carry what would change it.

### 1. The Decision entity gains three fields:

- `what_would_change_this` : the evidence or outcome that would reverse the recommendation. Required on every MEDIUM and HIGH risk proposal. A system that can state what would change its mind is a system a founder can safely delegate to.
- `outcome_label` : `approved` , `approved_edited` , `rejected` , `rejected_with_reason` , `more_evidence_requested` .
- `labeled_by` and `labeled_at` : provenance of the label.

2. **Every inbox action is captured as a label.** Approvals teach routing. Rejections with reasons are the highest value records in the company. Request-more-evidence teaches the evidence engine what convincing looks like for each founder.
3. **Eval datasets.** Per task type: golden sets and rubric libraries, starting with the MVP domain (web apps, where verification is mechanical) and expanding to specs and plans, where the label source is the founder's own decisions. Inter-rater agreement between model reviewers and human reviewers is tracked from the first run.
4. **The corpus compounds.** Objective, context, recommendation, label, outcome. At ten thousand companies this is an evaluation asset that exists nowhere else.

5. **Hard rule (Book 3 §7):** labels never cross tenants without explicit founder opt-in.
6. **Cold start.** Labels do not exist for the first fifty companies. Until the corpus is real: golden sets are hand built from the Wizard of Oz runs (Book 4 §4), thresholds stay conservative, and the autonomy ladder does the protecting. Companies start at L1, so human review is the default while the eval system is young.
7. **Labels are signals, not truth.** Founders make bad calls. Every label is weighed against the eventual outcome recorded in the Transition record. A founder decision that repeatedly correlates with poor outcomes loses weight in the corpus. The evidence engine learns from results, not from confidence. Causal learning (Book 0 §14) applies to us too.
8. **Independent counterfactuals.** `what_would_change_this` is never written by the pipeline that made the recommendation. A separate reviewer model generates the falsification condition, so the system grades its own homework with someone else's pen.

*Version history: v0.3. §10 extended with cold start, outcome weighting and independent counterfactuals (bench round 3). v0.2: added §10 and the new Decision fields. v0.1: first draft, ADRs proposed.*

# Book 2: Engineering & Architecture — Production Build Addendum v0.4

Book 2 remains the engineering source of truth.

Purpose: turn the existing architecture into executable contracts while preserving implementation freedom where the Canon does not need to prescribe a technology choice.

## ADR status normalization

The existing ADRs are proposed because the source Book 2 says so. This addendum separates binding architectural constraints from implementation choices. The team must not treat a vendor, framework, or exact infrastructure product as canon unless Book 4 ratifies it or the CTO records it as an implementation choice.

| ADR                     | Binding MVP requirement                                                                                           | Decision boundary                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| ADR-1 Orchestration     | Durable task execution; tasks survive worker/process crashes, retries and scheduling.                             | Exact workflow engine is an implementation choice unless performance/security evidence requires a Canon decision. |
| ADR-2 State storage     | Transactional primary state plus append-only immutable event history and derived read models.                     | Exact Postgres schema/index strategy is CTO implementation detail within the contract.                            |
| ADR-3 Runtime isolation | Disposable isolated worker session, scoped filesystem, no shared credentials, allowlisted egress, no host access. | Runtime technology is implementation choice; isolation properties are binding.                                    |
| ADR-4 Model routing     | All inference through Model Registry; task-type routing, version pinning, fallback and regression gate.           | Specific model providers are not canonical.                                                                       |
| ADR-5 Tool execution    | All tools pass through Tool Gateway with allowlist, scoped credentials and policy check.                          | Specific tool implementations are implementation choice.                                                          |
| ADR-6 Policy            | No worker action executes without policy/authority evaluation.                                                    | Policy semantics defined below are binding.                                                                       |
| ADR-7 Verification      | Risk-tiered verification with recorded verdict and evidence.                                                      | Exact reviewer model is implementation choice; independence requirement is binding.                               |
| ADR-8 Memory            | Layered context and task-scoped retrieval; no full-company context dumps.                                         | Storage/search technology is implementation choice.                                                               |
| ADR-9 Frontend          | Dashboard-first web application; chat is secondary.                                                               | Framework is implementation choice.                                                                               |
| ADR-10 Tenancy          | Tenant isolation applies to all tenant-bearing resources, not only DB rows.                                       | Exact infrastructure topology is implementation choice.                                                           |
| ADR-11 Secrets          | Central secret authority; runtime injection; secrets absent from model context.                                   | Vault product is implementation choice.                                                                           |
| ADR-12 Prompt injection | External content is untrusted; cannot directly trigger tools; policy re-check; exfiltration controls.             | Exact detection mechanism may evolve; control properties are binding.                                             |

## Canonical policy contract

```
policy.evaluate(company_id, worker_id, role, action_type, target, risk_tier, autonomy_level, authority_snapshot, budget_state, environment, context_ref)
```

```
returns: decision {ALLOW | DENY | REQUIRE_APPROVAL}, risk_tier, required_authority, required_approver, policy_version, reason, conditions[], expires_at
```

Every DENY and REQUIRE\_APPROVAL decision is auditable. An allowed action receives scoped authorization for the execution context. Tool implementations never infer authority from model text.

## Canonical MVP data contract

Book 0 owns the conceptual backbone. Book 2 owns the implementation fields. The MVP implementation must support the following entities at minimum.

| Entity       | Minimum implementation fields                                                                                                                                                                     |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Company      | id, founder_id, name, stage, autonomy_level, budget_cap, risk_tolerance, status, created_at, updated_at                                                                                           |
| Objective    | id, company_id, statement, structured, priority, constraints, status, version, created_at, updated_at                                                                                             |
| DesiredState | id, company_id, objective_id, success_criteria, target_metrics, deadline, version                                                                                                                 |
| Organization | id, company_id, version, status, approved_by, approved_at                                                                                                                                         |
| Role         | id, company_id, name, responsibilities, capabilities, authority_policy_id                                                                                                                         |
| Capability   | id, company_id, name, requirements, risk_profile, status                                                                                                                                          |
| Worker       | id, company_id, role_id, capabilities, intelligence_source_id, authority_policy_id, cost_profile, status, performance_profile                                                                     |
| Task         | id, company_id, workstream_id, objective_id, owner_worker_id, inputs, expected_output, dependencies, tools, budget, deadline, verification_method, risk_tier, authority_policy_id, status         |
| Action       | id, company_id, task_id, worker_id, action_type, target, context_ref, authority_snapshot, risk_tier, policy_decision, tool_ref, budget_before, budget_after, result_ref, verification_ref, status |
| Decision     | id, company_id, problem, evidence_refs, recommendation, cost, risk, confidence, what_would_change_this, status, outcome_label, labeled_by, labeled_at, resolved_by, resolved_at                   |
| Evidence     | id, company_id, claim, source_ref, evidence_type, confidence, interpretation, collected_at                                                                                                        |
| Verification | id, company_id, task_id, action_refs, risk_tier, method, checks, reviewer_type, reviewer_id, verdict, confidence, defects, evidence_refs, version, created_at                                     |
| Outcome      | id, company_id, task_id, transition_id, expected_result, actual_result, metrics, confidence, recorded_at                                                                                          |
| Transition   | id, company_id, problem, evidence_refs, proposed_change, expected_result, cost, risk, reversibility, authority_check, approval, actual_result, confidence                                         |
| Resource     | id, company_id, resource_type, provider, allocation, cost_profile, status                                                                                                                         |
| Constraint   | id, company_id, type, value, priority, source, status                                                                                                                                             |
| Permission   | id, company_id, subject_type, subject_id, action_type, target_scope, conditions, status                                                                                                           |

## Canonical event envelope and catalog

```
event_id, event_type, event_version, company_id, aggregate_type, aggregate_id, aggregate_version,
actor_type, actor_id, authority_snapshot, policy_decision, correlation_id, causation_id, command_id,
idempotency_key, context_refs[], payload, payload_schema_version, source_service, created_at
```

Events are immutable. Corrections are new events. State may be materialized from the event history, but historical events are never edited.

Canonical MVP event families: company.created, state.changed; objective.created, objective.changed; organization.changed; worker.hired, worker.assigned, worker.reassigned, worker.replaced; task.created, task.started, task.blocked, task.failed, task.completed, task.verified; action.proposed, action.allowed, action.denied, action.executed; decision.created, decision.approved, decision.rejected; budget.changed, budget.threshold\_reached; verification.completed; outcome.recorded; transition.proposed, transition.approved, transition.executed; deployment.created, deployment.verified, deployment.failed, deployment.rolled\_back; incident.created, incident.resolved.

## Runtime, deployment, tenancy and replay contracts

- Worker runtime: disposable session, scoped workspace, CPU/memory/disk/time/process limits, no host access, allowlisted egress, scoped credentials, cleanup guarantee and artifact extraction policy.
- Tool Gateway: worker request → identity → policy → budget → target validation → tool execution → result sanitization → audit event.
- Deployment lifecycle: BUILD → TEST → PACKAGE → PREVIEW → VERIFY → APPROVAL IF REQUIRED → DEPLOY → HEALTH CHECK → SMOKE TEST → VERIFIED/LIVE; failure triggers rollback where possible and incident handling.
- Tenant isolation covers database rows, object storage, queues, caches, worker workspaces, event streams, memory, logs, evaluation data, backups, secrets, deployment targets and exports.
- Replay requires sufficient provenance for company state, decision, context, intelligence/model version, action, tool, result and verification.

## Verification and autonomy promotion contract

Verification verdicts: VERIFIED, REJECTED, REQUIRES\_REWORK, REQUIRES\_HUMAN, INCONCLUSIVE. Verification records must identify method, evidence, reviewer provenance and version.

Autonomy is scoped by worker + capability + action class + environment + risk tier. Promotion requires recent capability-specific evidence, acceptable first-pass and escaped-defect performance, no unresolved SEV-1, no policy violations, stable model behavior and founder approval for level changes under the current Canon. Automatic demotion may occur under pre-approved safety triggers.

## Security and failure invariants

- External content is untrusted and cannot directly trigger tools.
- Secrets never enter contexts that read untrusted content.
- Every meaningful action has identity, authority, policy decision, context provenance and audit.
- Global kill switch freezes all workers for a company or the platform.
- Budget breaker stops company work at the hard cap.
- HIGH-risk actions remain human gated regardless of autonomy level.
- Rollback paths exist for code, infrastructure, configuration and organizational state.